Tyk

Tyk Onboarding

Sr. Customer Solutions Architect

Custom Plugins

Exploring custom plugins in Tyk

Custom Plugins in Tyk

- Custom Plugins enable execution of specific custom code for unique tasks, beyond Tyk's standard middleware.
- Users can hook plugins into various phases of the API lifecycle.
- Tyk includes 7 hooks for custom plugins across the middleware execution order.

Custom Middleware

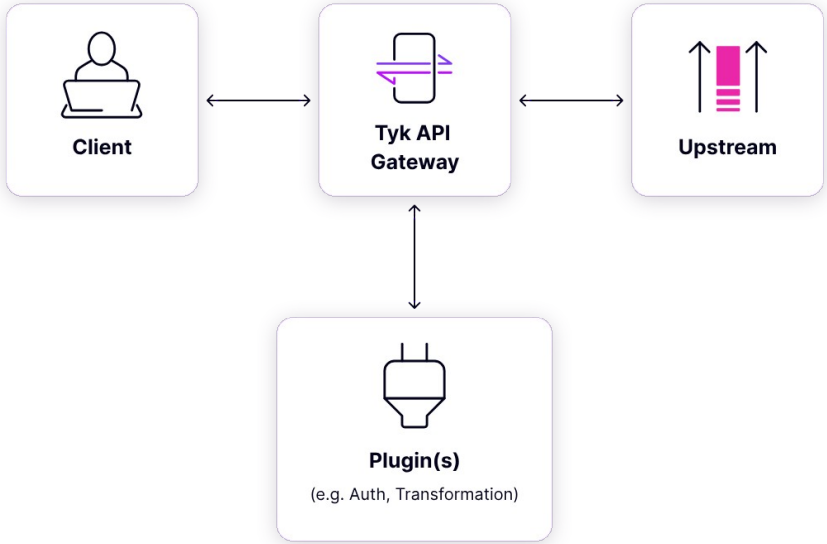
Hook Type	Plugin Type	Phase	Execution	Details	Common Use Cases
Pre (Request)	Request Plugin	HTTP Request	Before reverse proxy	First to execute, before middleware	IP rate limiting, request enrichment
Authentication	Auth Plugin	HTTP Request	Before reverse proxy	Replaces default authentication	Custom auth flow (e.g. legacy auth)
Post-Auth (Request)	Auth Plugin	HTTP Request	Before reverse proxy	After auth middleware	Special custom authentication logic
Post (Request)	Request Plugin	HTTP Request	Before reverse proxy	Final request phase middleware	Modify request (e.g., add headers)
Response	Response Plugin	HTTP Response	After reverse proxy	Executes after upstream API response	Modify response before returning to client
Analytics	Analytics Plugin	Request/Response	After reverse proxy	Final middleware for analytics	Obfuscate sensitive data in logs

Plugin Types

Plugin Type	Description
Go Plugins	Native plugins implemented in Go, the same language as Tyk Gateway, offering high performance.
gRPC Plugins	Executed remotely on a gRPC server, supporting development in any gRPC-compatible language.
JavaScript (JSVM)	JavaScript plugins run within an ECMAScript 5 compatible JavaScript Virtual Machine (JSVM).
Python Plugins	Embedded directly in Tyk Gateway, allowing Python code to run within the same process.

How it works

- The client sends request to API on Tyk Gateway.
- Tyk processes the request and forwards it to one or more plugins implemented and configured for that API.
- A plugin performs operations (e.g., custom authentication, data transformation).
- The processed request is then returned to Tyk Gateway, which forwards it upstream.
- Finally, the upstream response is sent back to the client.



Plugin Deployment Options in Tyk

- Local Plugins
  - The plugin source code and configuration are stored locally on the same file system as the Tyk Gateway.
  - Configuration is specified within the API Definition.
- Plugin Bundles (Remote)
  - Plugin source code and configuration are bundled into a zip file and uploaded to a remote web server.
  - Tyk Gateway downloads, extracts, caches, and executes these plugins for the configured phases of the API lifecycle.
- gRPC Plugins (Remote)
  - Custom plugins are hosted on a remote server and executed via gRPC.
  - Tyk Gateway sends requests to the gRPC server, allowing plugins to run at specific points during the API request/response lifecycle, using any preferred programming language.

Local Plugins

- Local Plugins
  - Pros:
    - Performance: Since the plugin code is stored locally, execution is fast with minimal latency.
    - Simplicity: Easy to configure and manage directly within Tyk Gateway.
    - No network dependencies: No need for external communication, making it ideal for environments without external connectivity.
  - Cons:
    - Limited scalability: Not suitable for large, distributed environments or for reusing plugins across multiple instances of Tyk Gateway.
    - Harder to manage updates: Every update requires manual changes on each Tyk instance.

Plugin Bundles

- Plugin Bundles
  - Pros:
    - Reusability: Multiple plugins can be bundled together and reused across different APIs and Tyk Gateways.
    - Centralized management: Plugins can be centrally stored and updated on a remote server, making it easier to manage changes.
    - Flexible deployment: Suitable for large, distributed architectures where plugins need to be updated across multiple gateways.
  - Cons:
    - Network dependency: Tyk Gateway depends on the remote server, which may introduce latency or fail if the server is down.
    - Complexity: Configuration and deployment can be more complex compared to local plugins.

gRPC Plugins

- gRPC Plugins (Remote)
  - Pros:
    - Language flexibility: You can write plugins in any gRPC-supported language, offering more flexibility for developers.
    - Scalability: Ideal for large-scale environments where plugins need to run across distributed systems.
    - Separation of concerns: Plugins are executed remotely, keeping the Tyk Gateway lightweight.
  - Cons:
    - Network latency: Involves remote execution, which can add network latency depending on the server's location and load.
    - More complex setup: Requires additional infrastructure (e.g., a gRPC server) and more configuration.
    - Potential failures: Relies on external gRPC server availability, which could lead to service disruptions if the server fails.

Plugin Development Flow

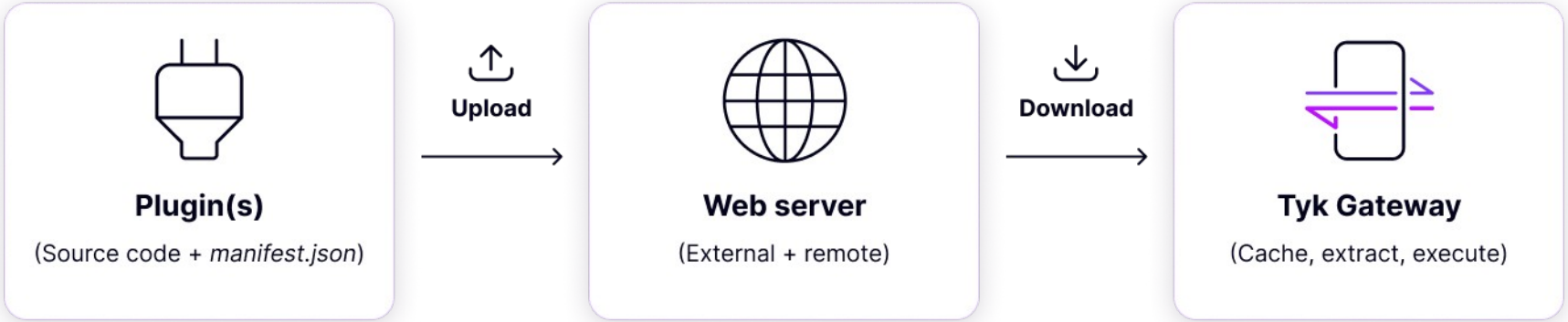
- Step 1: Initialize Go Module
  - Create a folder for your plugin.
  - Initialize Go modules - `go mod init tyk-plugin`
- Step 2: Write the Plugin
  - Plugin structure:
    - A Go main package.
    - An exported function like `AddFooBarHeader()` for custom logic.
- Step 3: Sync Dependencies
  - Run `go mod tidy`
- Step 4: Build the Plugin
  - Build the Go plugin as a shared library (.so) using Tyk's Docker image or Go commands.
    - ```
      docker run --rm -v `pwd`:/plugin-source \
      tykio/tyk-plugin-compiler:v5.2.1 plugin.so
      ```
      ```

Deploying Plugins Tyk Classic Def

- Step 1: Open Tyk Classic API Definition Editor
  - Select your API from the list of created APIs.
  - Click View Raw Definition to access the API Definition editor.
- Step 2: Edit API Definition
  - In the editor, navigate to the `custom_middleware` section.
  - Configure the plugins by updating this section with the necessary plugin settings.
- Step 3: Save Changes
  - Click Update to apply the changes to your Tyk Classic API Definition

```
            "custom_middleware": {
    "pre": [
        {
            "name": "myCustomPlugin",
            "path": "/opt/tyk-gateway/plugins/plugin.so"
            "driver": "otto/goplugin",
        }
    ]
}
```

Bundling Plugins



Bundling Plugins

- How Plugin Bundles Work
  - ZIP contains source code + manifest file.
  - API definition references the plugin bundle name.
  - Tyk Gateway downloads, caches, extracts, and executes plugins from the bundle.
- Bundle Caching
  - Plugin bundles are cached locally for efficiency.
  - Use unique filenames (e.g., version numbers) to update bundles.
  - Cache location: {TYK_ROOT}/{CONFIG_MIDDLEWARE_PATH}/bundles
- Gateway Configuration
  - Add the following to tyk.conf:
    - `"enable_bundle_downloader": true`
    - `"bundle_base_url": "http://my-bundle-server.com/bundles/"`
    - `"public_key_path": "/path/to/my/pubkey"`
    - `"enable_coprocess": true` (for rich plugins)

Bundler CLI Tool

- The Bundler tool is a CLI service, provided by the Gateway as part of its binary since v2.8
- To create plugin bundles, you will need
  - Manifest.json
  - Pugin source code files
  - Certificate key(optional)
- Use the following command to create a bundle:

```
$ tyk bundle build
```

- The resulting file will contain all your specified files and a modified manifest.json with the checksum and signature (if required) applied, in ZIP format.
- By default, Tyk will attempt to sign plugin bundles for improved security. If no private key is specified, the program will prompt for a confirmation. Use -y to override this

Bundles - Manifest Files

- Contains critical configuration details for the plugin bundle.
  - Lists the source code files to be included.
  - Files not specified in the manifest will not be included, even if present in the directory.
- Key Notes
  - `custom_middleware` block defines pre/post execution logic.
  - `driver` specifies the plugin language (e.g., Python).
  - `checksum` and `signature` are auto-filled during the build process.

Javascript custom plugin

- Step 1: Create a plugin within the gateway directory
  - injectCustomHeader.js

- Step 2: Modify the API definition on the Dashboard

```
"pre": [
        {
          "disabled": false,
          "name": "injectCustomHeader",
          "path": "/opt/tyk-gateway/jsvm/injectCustomHeader.js",
          "require_session": false,
          "raw_body_only": false
        }
```

- Test via postman

JSVM Custom plugin /w Bundles

- Step 1: Create 2 plugins within the gateway directory
  - injectCustomHeader.js
  - tracingHeader.js
- Step 2: Create a manifest.json file and place define both plugins in json file
- Step 3: Use tyk bundler CLI to create a bundle and serve it to the web server
- Step 4: Ensure you have a web server up and running
- Step 5: Update the API definition to receive plugins via the bundle

```
"custom_middleware_bundle": "bundle.zip",
```

- Step 6: Test via postman